目录

UFUG2106 —Round 4.1 双次本地复现实验报告 **1**
1. 研究目标与实验定义 1
2. 数学与算法依据 2
3. 实验环境与复现证据 2
4. 两次运行的完整指标 3
5. 逐案例 Runtime（两个实验的平均，单位 ms） 3
6. Area 退化分析及重点案例 4
7. 两次测量稳定性、解释边界与结论 5
8. 归档约定与证据索引 5

UFUG2106 —Round 4.1 双次本地复现实验报告

实验日期： 2026-11-11 | **实验阶段：** Round 4.1 —Runtime Optimization | **数据来源：** 用户 WSL2 本地两次完整实验结果包

> **结论**。Strict、Balanced、Fast 三种配置在两次独立执行中均为 **25/25 PASS、25/25 Target Delay Met**。共 150 次官方案例测评全部成功。两次运行产生的同一配置 Netlist **逐字节相同**；只存在运行时间的测量波动。

1. 研究目标与实验定义

上一阶段 Round 4 已在 25 个公开 Stage 1 案例上达到 Area 总和 528,910，全部电路正确且满足目标延迟，但重复综合多份 BDD 候选及反复进行全图极性优化造成额外 Runtime。Round 4.1 不改变逻辑功能或合法 Gate 集，而是优化综合器执行中的重复计算与搜索预算。

令 C 表示输出的合法门级电路，目标是：

$$\min A(C), \qquad D(C) \le D_{\text{target}}, \qquad T_{\text{alg}} \le T_{\text{limit}}, \qquad C \equiv f.$$

这里 $A(C)$ 为各门面积之和，$D(C)$ 为关键路径延迟，T_{alg} 为综合程序运行时间。**正确性与测评 Target Delay 是两类不同的条件；Target Delay 不达标并不自动等于逻辑不正确**。 Round 4.1 本次两者全部达成。Area 总和是跨案例描述性指标，并非教师未公布的最终评分。

1.1 三种版本的设计差异

- **Strict**：缓存输入真值表和变量影响度，减少极性搜索冗余，但保留足够候选，使本次公开 25 案例与 Round 4 的结果相同。
- **Balanced**：进一步压缩极性搜索候选数量，降低 Runtime，可能错失少量面积优化机会。
- **Fast**：在缓存及极性搜索削减之外进一步裁剪 BDD / 综合候选，优先降低 Runtime，允许更多 Area 损失。

以上不是三个不同的正确性标准；三种策略均通过相同的合法性与等价性验证。差异在于搜索预算与候选集。

2. 数学与算法依据

2.1 Truth Table / Influence 缓存

对第 i 个输入变量 x_i，设 f_j 为第 j 个输出。变量影响度可以定义为:

$$I_i = \sum_{x:x_i=0} \sum_{j=0}^{m-1} \mathbf{1}\left[f_j(x) \neq f_j(x \oplus e_i)\right].$$

其中 e_i 表示仅翻转变量 x_i 的二进制掩码。影响度由 Truth Table 唯一决定，与后续某个门级映射候选的参数无关，因此可以在一轮综合过程中计算一次、作为只读数据共享。

如果有 k 组综合候选，重复计算影响度需要 $O(knm2^n)$ 次布尔比较；缓存后该部分降至 $O(nm2^n)$，加上候选读取缓存的额外开销。此复杂度只描述影响度计算，**不意味着整个 BDD 综合算法降为线性时间**。存储解码后的 Truth Table 需要 $O(m2^n)$ 空间。

2.2 搜索空间缩减不影响布尔等价性

Shannon 展开及后续 De Morgan、吸收律、分配律重写属于恒等变换。以 Shannon 恒等式为例:

$$f = \neg x_i \wedge f_{x_i=0} \ \vee \ x_i \wedge f_{x_i=1}.$$

由 $x_i = 0$ 与 $x_i = 1$ 分别代入即可验证等价。搜索时舍弃部分候选，**不会破坏已保留候选的功能正确性**，只可能降低所能找到的面积/延迟最优程度。最终电路的有效性仍需独立验证，不能从启发式选择本身推出。

2.3 三维权衡和实验意义

优化问题实际上同时涉及 $A(C)$、$D(C)$ 与 T_{alg}。当全部候选满足 Timing Target 时，减少更多搜索步骤可以大幅降低综合用时；但这一收益未必以同样比例反映在 Area。Strict、Balanced、Fast 代表这一 Area–Runtime Trade-off 的三组经验设计点，而**不是全局 Pareto 最优的数学证明**。

3. 实验环境与复现证据

记录	内容
Run 1	`20261111_121043_Round4.1_local_results.zip`, 北京时间 12:10:43
Run 2	`20261111_121216_Round4.1_local_results.zip`, 北京时间 12:12:16
系统	WSL2，Linux `6.6.87.2-microsoft-standard-WSL2`
处理器	Intel Core Ultra 9 275HX，24 个可见逻辑 CPU
编译器	GCC 13.3.0，`-O2 -std=c++17`
Python	3.12.3
Evaluator	课程给定 Linux Evaluator，SHA-256 两次一致
输入用例	两次均为相同的 25 个 Stage 1 Cases，文件哈希清单一致

记录	内容
输出目录	各运行的 `strict/`、`balanced/`、`fast/`、`source_snapshot/`

证据校验。两次运行的 Makefile、三份 C++ 源码及相关 Header SHA-256 全部相同，25 个输入文件的哈希清单一致。每个配置两次运行产生的 25 个 Netlist 都逐字节相同，共核对 75 对。进一步将 Strict 的 25 个 Netlist 与之前上传的 Round 4 本地输出比较，也全部逐字节一致。

独立验证器在每个配置、每次运行中均完成 **13 个小规模案例全输入穷举 + 12 个较大案例确定性抽样**，并重新核算 Area 与 Delay；全部通过，共 150 份独立验证记录。官方 Evaluator 同时对全部 150 次案例执行返回 `PASS`。注意：独立验证对较大案例不是全赋值穷举，故不可把其覆盖程度夸大为所有 Truth Table 的独立穷举。

4. 两次运行的完整指标

版本	PASS / Target Met	Area 总和	Run 1 Median (ms)	Run 2 Median (ms)	两次中位数均值 (ms)
Round 4 原版（历史运行）	25/25	528,910	127.568	—	—
Strict	25/25	**528,910**	111.149	117.527	**114.338**
Balanced	25/25	**529,014**	96.048	102.615	**99.332**
Fast	25/25	**530,822**	67.605	63.035	**65.320**

两次中位数均值是 $\frac{1}{2}(\text{median}(T_1) + \text{median}(T_2))$，不是把 50 个观测直接混在一起后取中位数。原 Round 4 只提供此前的同机历史运行，不应视为本次试验中同步重复的控制组。

版本	Area 相对 Round 4 变化	Median Runtime 相对历史 Round 4 的变化	两次 Max Runtime (ms)	两次 Runtime 总和 (ms)
Strict	+0 (+0.000%)	下降约 10.4%	3337.7 / 3084.0	10270.8 / 10024.7
Balanced	+104 (+0.020%)	下降约 22.1%	2697.4 / 2699.4	9081.4 / 9062.7
Fast	+1,912 (+0.361%)	下降约 48.8%	2299.3 / 2273.6	7236.4 / 7422.6

解释：上表时间变化为同一机器、不同时间的经验比较，不是同批次随机化交叉试验；Windows `/mnt/e` 文件系统 I/O、CPU 动态频率和后台负载均可能产生影响。两次运行仍然证明了各策略 Runtime 收益的总体方向具有一定重复性。

5. 逐案例 Runtime（两个实验的平均，单位 ms）

以下表中的时间是同一配置、同一案例两次运行的算术平均；它描述实际花费，并不意味着每个案例只执行了一次。Area 与 Delay 均取两次完全一致的测量值。

Case	Round 4	Strict	Balanced	Fast	Fast / Round 4
001	1013.3	982.0	843.0	734.9	0.73×

Case	Round 4	Strict	Balanced	Fast	Fast / Round 4
002	371.3	338.7	315.0	269.5	0.73×
003	405.0	376.3	350.3	302.1	0.75×
004	45.6	38.9	33.6	25.3	0.56×
005	45.3	47.7	42.6	32.3	0.71×
006	56.5	52.3	40.8	29.5	0.52×
007	1553.2	1211.7	1052.1	696.6	0.45×
008	86.2	67.2	56.9	41.3	0.48×
009	127.6	114.3	99.3	65.3	0.51×
010	231.4	154.2	148.3	107.9	0.47×
011	48.4	47.9	49.4	38.4	0.79×
012	46.2	45.0	39.8	30.5	0.66×
013	170.3	162.9	147.5	124.3	0.73×
014	529.3	376.8	378.7	223.5	0.42×
015	667.7	583.4	556.2	465.3	0.70×
016	28.7	26.4	23.7	18.5	0.64×
017	11.8	10.4	10.7	8.3	0.70×
018	24.6	20.8	17.3	13.7	0.56×
019	245.3	190.5	189.3	156.9	0.64×
020	46.8	45.9	45.0	35.8	0.77×
021	675.3	632.0	610.5	548.4	0.81×
022	58.1	59.1	55.6	40.0	0.69×
023	12.3	12.2	15.2	9.5	0.77×
024	3536.0	3210.9	2698.4	2286.4	0.65×
025	1408.6	1340.3	1252.8	1025.5	0.73×

从上表看，大型案例仍然耗时显著。例如 `case_024` 的两次 Fast 平均仍约 2.29 秒；`case_025` 约 1.03 秒。优化并没有消除大实例的主要综合复杂度，只是削减了可避免的重复搜索。

6. Area 退化分析及重点案例

Area 的变化不是均匀的：Balanced 相对 Strict 只有 10 个案例出现面积变化，累计增加 104；Fast 相对 Strict 有 14 个案例面积变化，累计增加 1,912。未变化的案例仍应保留在逐案例 CSV 中，以供最终报告检查。

Case	Strict Area	Balanced Area	Fast Area	Fast−Strict
018	1,231	1,232	2,463	+1,232
007	78,042	78,042	78,348	+306
014	11,634	11,662	11,808	+174
009	14,584	14,584	14,661	+77
019	5,319	5,319	5,353	+34
003	24,236	24,251	24,264	+28
022	1,302	1,320	1,318	+16
005	700	700	715	+15
011	452	464	462	+10
017	428	440	435	+7
016	2,149	2,149	2,154	+5
023	123	125	128	+5
006	5,962	5,962	5,964	+2
024	124,466	124,475	124,467	+1

Case	Strict Area	Balanced Area	Fast Area	Fast−Strict
013	11,248	11,252	11,248	+0
015	33,371	33,374	33,371	+0

主要反例：`case_018`。 Fast 的 Area 从 1,231 增至 2,463（增加 1,232，约翻倍），这一个案例就贡献了 Fast 总 Area 退化的约 64.4%。因此 Fast 不应被描述为“每个案例只损失少量 Area”；它只是**整个公开案例集合的总 Area 损失较小**。

同时，Strict 在本次 25 个公开案例上与原 Round 4 保持相同 Area 和 Delay，且输出 Netlist 完全一致。这是“重复计算消除与较温和搜索预算裁剪能够保留既有输出”的直接实验依据，但不保证对未知案例也必然字节一致。

7. 两次测量稳定性、解释边界与结论

相比 Run 1，Run 2 的中位 Runtime 并非对每个版本都单调变小：Strict 为 111.149→117.527 ms；Balanced 为 96.048→102.615 ms；Fast 为 67.605→63.035 ms。由于每次运行的顺序固定为 Strict→Balanced→Fast，且两次之间没有随机化版本顺序，可能存在缓存/系统负载/温度的顺序效应。**仅有两次测量不足以建立严谨置信区间或硬件无关的加速保证**。

7.1 推荐冻结方案

建议将 Balanced 作为 Round 4.1 默认研究候选，Strict 保留为面积对照，Fast 保留为运行时间对照。理由是 Balanced 的总 Area 只比 Strict 增加 104（约 0.020%），但两次运行的中位时间分别降至 96.048 与 102.615 ms；Fast 在本地中位时间更低，但 `case_018` 等局部 Area 退化需要明确记录。

若最终评分对 Area 赋予极高权重，Strict 仍有价值；若 Runtime 限制更紧，Fast 可能更有优势。由于教师尚未公布完整评分函数，不能直接证明任一档具有最高分。

7.2 下一轮研究方向

1. **BDD 构造缓存：**对相同 Variable Order 的不同 Gate Mapping 候选复用 BDD，减少 k 次结构重复构建。这是更强的公共子问题消除，不只是缓存 Truth Table。
2. **增量成本评估：**对局部极性变更和 DAG 重写维护受影响 Gate 的 Area / Arrival Time，减少每次全图重算。
3. **条件式搜索预算：**根据 BDD 节点数、已找到候选的 Area、Timing Slack 和边际收益调整候选数量，而非固定每次遍历相同集合。
4. **更严格的中断机制：**对单个候选的构造、后处理均增加超时及节点数量检查，以避免在未知高熵函数上超时。
5. **更科学的性能基准：**建议在 WSL 原生文件系统再跑一组，随机化三种配置运行顺序并增加重复次数，区分真实算法收益与 I/O/负载影响。

8. 归档约定与证据索引

本报告建议放在 `local_runs/YYYYMMDD_HHMMSS_Round4.1/report_ready/`，但保留原来两个本地运行 ZIP 不变。两份原始 ZIP、官方 CSV、原始 Netlist、验证器输出与源码快照是本报告的底层证据；本报告和逐案例合并 CSV 是二次分析产物，不应覆盖它们。

建议同时存档：

- `Round4.1_TwoRun_Local_Reproduction_Report_20261111.md`（本报告）
- `Round4.1_TwoRun_Local_Reproduction_Report_20261111.pdf`（可提交/分享的阅读版）
- `Round4.1_TwoRun_PerCase_Comparison.csv`（25 案例 × 3 配置 × 2 次运行）
- `Round4.1_TwoRun_Stats.json`（机器可读统计与源文件哈希）

8.1 Source Provenance（完整 SHA-256 见 JSON）

- 两次使用的 Stage 1 输入哈希清单 SHA-256 前 16 位：`923b4f333cf1ebc6`…。
- `Makefile`：`bcdd7a681940e118`…。
- `src/synth_strict.cpp`：`a447a6d3e29c29d0`…。
- `src/synth_balanced.cpp`：`906d250fd3384d84`…。
- `src/synth_fast.cpp`：`62c9e906096546b6`…。
- `src/algebraic_mapper.hpp`：`33d9d02a4fe2cbb1`…。
- `src/polarity_mapper.hpp`：`46e8183e644071ca`…。

声明： 所有实验数据来自两份用户提供的 WSL ZIP 和先前用户提供的冻结 Round 4 WSL ZIP。数学部分是对归档源码与算法文档的解释，不表示这些公式本身经过独立的形式化证明器验证。